

Async SQLAlchemy & Connection Pooling

ML Engineer Journey | Phase 4 — Database & Observability | 12 Jun 2026

1. Kenapa Async?

Python secara default adalah *synchronous* — satu baris selesai, baru baris berikutnya berjalan. Masalah muncul saat baris itu adalah query database: CPU harus **berdiri diam** menunggu PostgreSQL mengembalikan data. Untuk satu request tidak terasa. Untuk ratusan request bersamaan — ini bottleneck serius.

Analogi: Kamu adalah pelayan restoran.

Sync: ambil pesanan meja 1, pergi ke dapur, **berdiri diam menunggu** masak selesai, baru layani meja 2.

Async: titip pesanan ke dapur, **langsung balik** ke meja 2. Saat dapur panggil 'siap!', baru antarkan. CPU tidak pernah nganggur.

Dengan **async I/O**, FastAPI bisa suspend coroutine yang sedang menunggu DB, handle request lain dulu, lalu kembali saat query selesai — semua dalam satu thread via Python asyncio event loop.

2. Sync vs Async SQLAlchemy

Aspek	Sync SQLAlchemy	Async SQLAlchemy
Engine	create_engine()	create_async_engine()
Session	Session	AsyncSession
Query	db.query(Model)	select(Model) + await db.execute()
Get by PK	db.get(M, pk)	await db.get(M, pk)
Commit	db.commit()	await db.commit()
Flush	db.flush()	await db.flush()
Lazy load	Bekerja otomatis	MissingGreenlet error
expire_on_commit	True (aman)	Wajib False
URL driver	postgresql://	postgresql+asyncpg://
Context manager	with Session()	async with AsyncSession()

3. Setup: database.py

Install driver async: pip install sqlalchemy[asyncio] asyncpg. Driver **asyncpg** adalah pengganti psycopg2 untuk context async — murni async, tanpa threading.

database.py — kode lengkap:

```
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.orm import DeclarativeBase
```

```

from sqlalchemy.ext.asyncio import async_sessionmaker

SYNC_URL = os.getenv('DATABASE_URL', 'postgresql://user:pw@localhost/db')
ASYNC_URL = SYNC_URL.replace('postgresql://', 'postgresql+asyncpg://')

engine = create_async_engine(
    ASYNC_URL,
    pool_pre_ping=True,    # test koneksi sebelum pakai
    pool_size=10,          # koneksi permanen siaga
    max_overflow=20,       # koneksi tambahan saat pool habis
    pool_timeout=30,       # detik tunggu koneksi tersedia
    pool_recycle=1800,     # recycle tiap 30 menit
    echo=False,
)

SessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False # wajib False di async
)

async def get_db():
    async with SessionLocal() as db:
        try:
            yield db
            await db.commit() # auto-commit jika endpoint sukses
        except Exception:
            await db.rollback() # rollback hanya saat error
            raise

```

expire_on_commit=False — wajib di async. Secara default SQLAlchemy meng-expire semua attribute object setelah commit, memaksanya di-reload dari DB saat diakses. Di sync ini bekerja via lazy load otomatis. Di async, lazy load memerlukan await — tapi Python tidak mengizinkan await saat mengakses attribute biasa (obj.title). Hasilnya: **MissingGreenlet error**. Solusi: matikan expire dengan False.

try/except vs finally di get_db(): rollback harus di except, bukan finally. finally selalu berjalan — termasuk saat sukses — sehingga akan membatalkan commit yang baru selesai. async with SessionLocal() sudah otomatis close session di akhir blok.

4. Endpoint Async — Pola Kunci

Operasi	Sync (lama)	Async (baru)
List all	db.query(M).all()	stmt=select(M); (await db.execute(stmt)).scalars().all()
Get by PK	db.get(M, pk)	await db.get(M, pk)
Insert + PK	db.add(); db.flush()	db.add(); await db.flush()
Partial update	setattr loop	setattr loop (sama, hanya context async)
Delete	db.delete(obj)	await db.delete(obj)
Refresh	db.refresh(obj)	await db.refresh(obj)

`.scalars().all()` — await `db.execute()` mengembalikan `CursorResult`, bukan list. `.scalars()` mengambil kolom pertama (ORM object), `.all()` mengkonversi ke list Python.

5. Connection Pooling

Membuka koneksi baru ke PostgreSQL ada overhead: TCP handshake, autentikasi, alokasi memori di server. Connection pool menyimpan koneksi yang sudah terbuka dalam kondisi siaga — request langsung pakai, tidak perlu buka koneksi baru setiap saat.

Analogi: Pool taksi siaga di depan kantor.

Tanpa pool: setiap request panggil taksi dari nol — menunggu, overhead tinggi.

Dengan pool: taksi sudah parkir siap berangkat. Selesai antar, kembali ke pool — tidak pulang ke garasi.

Parameter	Default	Fungsi
<code>pool_size</code>	5	Koneksi permanen yang selalu hidup di pool
<code>max_overflow</code>	10	Koneksi tambahan saat <code>pool_size</code> habis (sementara)
<code>pool_timeout</code>	30	Detik tunggu koneksi tersedia, lalu <code>TimeoutError</code>
<code>pool_recycle</code>	-1	Recycle koneksi setelah N detik (set 1800 di prod)
<code>pool_pre_ping</code>	False	Test koneksi sebelum pakai — wajib True di prod

Sizing formula:

```
total koneksi per instance = pool_size + max_overflow = 10 + 20 = 30
total ke DB (4 instance)   = 30 x 4 = 120 koneksi
```

Implikasi: PostgreSQL default `max_connections` = 100. Dengan 4 instance x 30 = 120 koneksi maksimum, kamu **melebihi batas**. Solusi: turunkan `pool_size`, kurangi instance, atau gunakan **PgBouncer** (connection pooler di level infra yang menggabungkan koneksi dari banyak app ke pool tunggal ke DB).

6. Jebakan Async: Lazy Loading

Relationship yang belum di-set `lazy='selectin'` akan menjadi masalah di async. Lazy load mencoba hit database saat attribute diakses — tapi di luar async context, tidak ada `await` yang bisa dipakai. Solusinya: eager load saat query.

```
# BAHAYA - MissingGreenlet
result = await db.get(AnalysisRequest, uid)
return result.scenarios # lazy load di sini = ERROR

# BENAR - eager load dengan selectinload
from sqlalchemy.orm import selectinload
stmt = (
    select(AnalysisRequest)
    .options(selectinload(AnalysisRequest.scenarios))
    .where(AnalysisRequest.id == uid)
)
result = await db.execute(stmt)
return result.scalar_one_or_none()
```

Jika model sudah di-set lazy='selectin' di definisi relationship, SQLAlchemy otomatis eager load setiap kali query — tidak perlu selectinload() manual.

7. DatabaseManager untuk CLI Scripts

FastAPI pakai get_db() async. Script CLI (misal: seed data, migration script) tidak berjalan dalam async context — mereka butuh sync session. Jangan campur keduanya.

```
# Sync engine terpisah khusus CLI
sync_engine = create_engine(SYNC_URL, pool_pre_ping=True)

class DatabaseManager:
    def __init__(self):
        self._factory = sessionmaker(bind=sync_engine)

    @contextmanager
    def session(self):
        s = self._factory()
        try:
            yield s
            s.commit()
        except Exception:
            s.rollback()
            raise
        finally:
            s.close() # sync: finally untuk close adalah benar

db_manager = DatabaseManager()
```

Quick Reference — Checklist Migrasi Sync → Async

Checklist	Status
URL: postgresql:// → postgresql+asyncpg://	Wajib
create_engine → create_async_engine	Wajib
sessionmaker → async_sessionmaker	Wajib
Session → AsyncSession	Wajib
expire_on_commit=False	Wajib
def get_db → async def get_db	Wajib
db.query(M) → select(M) + await db.execute()	Wajib
db.commit/flush/delete → await db.commit/flush/delete	Wajib
pool_size, max_overflow, pool_recycle, pool_pre_ping	Recommended
Semua relationship: lazy='selectin' atau selectinload()	Recommended
DatabaseManager sync terpisah untuk CLI scripts	Best practice